

INGENIERÍA DE SOFTWARE 1

Informe de Modulo de Testing

No Pain No Main()



Autores: Andres Sebastian Sanchez Quintero

Yohan Steven Jiménez Hilarión

Miguel Angel Hincapie Santacruz

Sebastian Fernando Gutiérrez Rojas

Julian Ramirez Diaz

14/06/2026

1. Introducción

El presente informe documenta las pruebas unitarias realizadas sobre la capa de servicios del proyecto **NoPainNoMain**. El objetivo del proceso de testing es verificar que cada componente de la lógica de negocio se comporte correctamente de forma aislada, garantizando robustez frente a entradas inesperadas y condiciones límite.

Se empleó el framework **JUnit 5** junto con **Mockito** para la creación de dobles de prueba (*mocks*), de modo que cada clase se prueba de forma independiente, sin depender de la base de datos ni de otros servicios en ejecución.

Las pruebas abarcan **12 métodos** distribuidos en **7 clases** de la capa de servicios, cubriendo los flujos principales de disponibilidad de máquinas, registro, confirmación y cancelación de reservas.

2. Cobertura de las Pruebas

2.1. Qué significa la cobertura?

La cobertura de pruebas (*test coverage*) es una métrica que indica qué proporción del código fuente es ejecutada durante la ejecución de los tests. Su propósito no es garantizar que el software esté libre de errores, sino confirmar que las rutas más importantes de la lógica han sido ejercidas y validadas.

- **Cobertura de líneas:** porcentaje de líneas de código ejecutadas al menos una vez.
- **Cobertura de ramas:** porcentaje de bifurcaciones (`if/else`, `switch`) cubiertas en ambos sentidos (verdadero y falso).
- **Cobertura de métodos:** porcentaje de métodos invocados al menos una vez.

Una cobertura alta es deseable, pero no suficiente por sí sola: también importa la *calidad* de las aserciones. Por eso este informe complementa los números con la descripción de los **casos límite** probados.

2.2. Resumen de cobertura por clase

La siguiente tabla resume la cobertura lograda sobre los métodos bajo prueba. Los valores corresponden a la proporción de escenarios funcionales cubiertos respecto al total de escenarios identificados en el análisis.

Clase	Método bajo prueba	Tests	Cobertura
AvailabilityService	findFreeSlotsByMachine	5	Alta
BookingQuery	hasOverlappingBooking	6	Alta
AdministratorRegister	save(String)	4	Alta
BookingRegister	save(String)	2	Alta
	save(Booking)	9	Alta
AdministratorRegisterHelper	isDuplicateDocument()	3	Alta
	isDuplicateEmail()	3	Alta
BookingRegisterHelper	isSlotAlreadyPassed	6	Alta
	hasReachedActiveLimit	5	Alta
BookingCancelService	cancel(String)	6	Alta
BookingConfirmService	confirm(String)	4	Alta
	releaseExpiredSlots()	6	Alta
BookingConfirmHelper	findTodayActiveBookings	4	Alta
	findBookingReadyToStart	6	Alta
	hasNextBooking	4	Alta

```

[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.adanext.NoPainNoMain.service.query.AvailabilityServiceTest
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org.mockito/Mockito.html#0.3
WARNING: A Java agent has been loaded dynamically (C:\Users\PC\m2repository\net\bytebuddy\byte-buddy-agent\1.17.8\byte-buddy-agent-1.17.8.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disabled by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.116 s -- in com.adanext.NoPainNoMain.service.query.AvailabilityServiceTest
[INFO] Running com.adanext.NoPainNoMain.service.query.BookingQueryTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.049 s -- in com.adanext.NoPainNoMain.service.query.BookingQueryTest
[INFO] Running com.adanext.NoPainNoMain.service.register.AdministratorRegisterTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.169 s -- in com.adanext.NoPainNoMain.service.register.AdministratorRegisterTest
[INFO] Running com.adanext.NoPainNoMain.service.register.BookingRegisterTest
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.137 s -- in com.adanext.NoPainNoMain.service.register.BookingRegisterTest
[INFO] Running com.adanext.NoPainNoMain.service.register.helpers.AdministratorRegisterHelperTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.019 s -- in com.adanext.NoPainNoMain.service.register.helpers.AdministratorRegisterHelperTest
[INFO] Running com.adanext.NoPainNoMain.service.register.helpers.BookingRegisterHelperTest
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.067 s -- in com.adanext.NoPainNoMain.service.register.helpers.BookingRegisterHelperTest
[INFO] Running com.adanext.NoPainNoMain.service.update.BookingCancelServiceTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.001 s -- in com.adanext.NoPainNoMain.service.update.BookingCancelServiceTest
[INFO] Running com.adanext.NoPainNoMain.service.update.BookingConfirmServiceTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.226 s -- in com.adanext.NoPainNoMain.service.update.BookingConfirmServiceTest
[INFO] Running com.adanext.NoPainNoMain.service.update.helpers.BookingConfirmHelperTest
[INFO] Tests run: 14, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.090 s -- in com.adanext.NoPainNoMain.service.update.helpers.BookingConfirmHelperTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 73, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 5.629 s
[INFO] Finished at: 2026-06-14T18:12:13-05:00
[INFO] -----
PS C:\Users\PC\Documents\Wo-Pain-No-Pain\project-backend\NoPainNoMain>

```

En total se escribieron **73 pruebas unitarias** sobre los 15 métodos objetivo. La calificación “Alta” indica que todos los caminos de ejecución principales incluyendo las ramas de error y los casos límite identificados cuentan con al menos un test que los valida.

3. Casos Límite por Clase y Método

A continuación se describe, para cada método bajo prueba, el conjunto de casos de prueba implementados con énfasis en los **casos límite** (*edge cases*): aquellas condiciones extremas o poco frecuentes donde la lógica es más propensa a fallar.

3.1. AvailabilityService findFreeSlotsByMachine

Este método devuelve la lista de turnos horarios disponibles para una máquina en una fecha concreta, sustrayendo los turnos ya reservados.

Casos de prueba

- TC1. **Sin reservas existentes** se devuelven todos los turnos disponibles.
- TC2. **Algunos turnos reservados** solo se devuelven los turnos libres; los reservados quedan excluidos correctamente.
- TC3. **Todos los turnos reservados** la lista de respuesta está vacía.
- TC4. **Reservas de otras máquinas** las reservas de máquinas distintas no afectan la disponibilidad de la máquina consultada.
- TC5. **[Límite] Múltiples reservas sobre el mismo turno** el turno aparece excluido *una sola vez*; no se producen duplicados en la lista resultado.

Relevancia del caso límite TC5: si el método calculara la diferencia sin eliminar duplicados, un turno podría aparecer ausente de la lista libre incluso cuando aún debería estar disponible, o causaría errores de conteo. La prueba garantiza idempotencia en la exclusión.

3.2. BookingQuery hasOverlappingBooking

Detecta si un estudiante ya tiene una reserva activa en el mismo turno y la misma fecha que la nueva reserva solicitada.

Casos de prueba

- TC1. **[Límite] Reserva nula** devuelve `false` sin lanzar excepción.
- TC2. **Sin reservas previas del estudiante** no hay solapamiento (`false`).
- TC3. **Misma fecha y mismo turno** solapamiento detectado (`true`).
- TC4. **Misma fecha, turno diferente** no hay solapamiento (`false`).
- TC5. **Mismo turno, fecha diferente** no hay solapamiento (`false`).
- TC6. **Múltiples reservas previas; solo una coincide** solapamiento detectado correctamente (`true`).

Relevancia del caso límite TC1: una reserva nula podría llegar a este método si hay un error previo en la cadena de llamadas. Retornar `false` de forma segura en lugar de lanzar un `NullPointerException` hace que el sistema falle de forma controlada.

3.3. AdministratorRegister save(String)

Se encarga de deserializar el JSON de entrada, aplicar las validaciones de unicidad de documento y correo electrónico, y finalmente persistir al nuevo administrador.

Casos de prueba

- TC1. JSON válido y datos únicos** se guarda exitosamente y retorna la entidad persistida.
- TC2. [Límite] Documento duplicado** lanza `IllegalStateException` y aborta el flujo sin verificar el correo ni guardar.
- TC3. [Límite] Correo duplicado** lanza `IllegalStateException` y prohíbe la persistencia.
- TC4. [Límite] JSON con formato inválido** lanza `IllegalArgumentException` deteniendo la ejecución antes de consultar reglas de negocio.

Relevancia de los casos límite TC2 y TC4: Demuestran la estrategia de *falla rápida* (fail-fast) y de aislamiento. Si el formato del JSON es incorrecto, el sistema no intenta contactar a la base de datos ni a los validadores. Igualmente, si el documento ya existe, se ahorra la consulta del correo electrónico, optimizando el uso de recursos transaccionales.

3.4. BookingRegister save(String)

Versión del guardado que recibe la reserva como cadena JSON, la convierte al dominio y delega en el método `save(Booking)`.

Casos de prueba

- TC1. JSON válido** se convierte y guarda; se retorna el objeto persistido.
- TC2. Generación del ID compuesto** el identificador tiene el formato `fecha_maquina_turno`.

Relevancia: esta variante prueba el flujo de integración JSON→dominio sin duplicar todas las validaciones ya cubiertas en `save(Booking)`, verificando que la conversión y la generación del ID funcionan correctamente.

3.5. BookingRegister save(Booking)

Núcleo del registro de reservas. Aplica una cadena de validaciones antes de persistir y actualizar el estado de la máquina.

Casos de prueba

- TC1. Reserva válida** se guarda y se actualiza el estado de la máquina a *reservada*.
- TC2. [Límite] Reserva nula** `IllegalArgumentException`; no se llama al repositorio.
- TC3. Fecha pasada** `IllegalStateException`.
- TC4. Turno ya transcurrido** `IllegalStateException`.
- TC5. Reserva en fin de semana** `IllegalStateException`.
- TC6. Fecha fuera de la semana actual** `IllegalStateException`.
- TC7. [Límite] ID duplicado** `IllegalStateException`; garantiza idempotencia.
- TC8. Límite activo alcanzado** `IllegalStateException`.
- TC9. Solapamiento de horarios** `IllegalStateException`.

Relevancia de TC7 (ID duplicado): el identificador de una reserva se construye como `fecha_máquina_turno`, lo que lo hace naturalmente único. Sin embargo, si por alguna razón llegara una solicitud duplicada, el sistema debe rechazarla explícitamente en lugar de sobrescribir silenciosamente un registro existente.

3.6. `AdministratorRegisterHelper isDuplicateDocument`

Verifica de manera aislada si el número de documento proporcionado ya se encuentra registrado en la base de datos de administradores.

Casos de prueba

- TC1. El documento ya existe retorna `true`.
- TC2. El documento no existe retorna `false`.
- TC3. **[Límite]** Documento nulo retorna `false` de forma segura sin realizar consultas a la base de datos.

Relevancia del caso límite TC3: Al aplicar una evaluación de cortocircuito lógico (*short-circuiting*), el método evita propagar un `null` hacia la capa del repositorio JPA, previniendo excepciones inesperadas (`NullPointerException`) y ahorrando tiempo de procesamiento en el motor de persistencia.

3.7. `AdministratorRegisterHelper isDuplicateEmail`

Verifica de manera independiente si la dirección de correo electrónico ya ha sido registrada por otro administrador en el sistema.

Casos de prueba

- TC1. El correo electrónico ya existe retorna `true`.
- TC2. El correo electrónico no existe retorna `false`.
- TC3. **[Límite]** Correo electrónico nulo retorna `false` inmediatamente sin consultar el repositorio.

Relevancia del caso límite TC3: Al igual que con el documento, validar la nulidad antes de delegar la búsqueda en JPA protege el sistema contra datos parciales o malformados, asegurando que el servicio falle de forma predecible y controlada.

3.8. `BookingRegisterHelper isSlotAlreadyPassed`

Determina si el turno horario de una reserva ya transcurrió, combinando la fecha y la hora de inicio del turno con el instante actual.

Casos de prueba

- TC1. Hora de inicio en el futuro el turno no ha pasado (`false`).
- TC2. Hora de inicio en el pasado el turno ya pasó (`true`).
- TC3. Fecha futura con hora pasada si la fecha es posterior a hoy, el turno no ha pasado independientemente de la hora (`false`).
- TC4. **[Límite]** Turno sin `startTime` pero con `id` el método resuelve el turno desde el repositorio y evalúa correctamente.
- TC5. **[Límite]** Fecha nula en la reserva devuelve `false` de forma segura.
- TC6. **[Límite]** Turno nulo en la reserva devuelve `false` de forma segura.

Relevancia de TC4: en escenarios donde la reserva proviene de una deserialización

parcial, el objeto `TimeSlot` puede tener solo el identificador. El método debe poder recuperar los detalles desde la base de datos para funcionar correctamente. TC5 y TC6 protegen contra datos incompletos que de otro modo causarían `NullPointerException`.

3.9. `BookingRegisterHelper` `hasReachedActiveLimit`

Verifica si un estudiante ha alcanzado el número máximo de reservas activas permitidas simultáneamente.

Casos de prueba

- TC1. **Estudiante por debajo del límite** puede reservar (`false`).
- TC2. **[Límite] Estudiante exactamente en el límite** no puede reservar (`true`).
- TC3. **[Límite] Estudiante por encima del límite** no puede reservar (`true`).
- TC4. **[Límite] Estudiante nulo** devuelve `false` sin excepción.
- TC5. **[Límite] Número de documento nulo** devuelve `false` sin excepción.

Relevancia de TC2: la comparación $\geq$ frente a $>$ marca la diferencia entre permitir o bloquear una reserva cuando se alcanza el límite justo. Un error en este operador pasaría desapercibido sin esta prueba específica. TC4 y TC5 aseguran tolerancia a datos de entrada mal formados.

3.10. `BookingCancelService` `cancel(String)`

Gestiona la cancelación de una reserva, liberando la máquina asociada si la operación se realiza con suficiente antelación.

Casos de prueba

- TC1. **Reserva cancelable** se cancela, se libera la máquina y se persiste el cambio.
- TC2. **[Límite] ID inexistente** `IllegalStateException`; no se modifica nada.
- TC3. **[Límite] Cancelación demasiado próxima al inicio** `IllegalStateException` con mensaje que indica los minutos mínimos de antelación.
- TC4. **Mensaje de error incluye los minutos de antelación** mejora la experiencia de usuario con información accionable.
- TC5. **Orden de operaciones** la máquina se libera *antes* de actualizar el estado de la reserva y persistirla.
- TC6. **Se retorna la versión actualizada desde el repositorio** consistencia de datos.

Relevancia de TC5 (orden de operaciones): si la reserva se marcara como cancelada antes de liberar la máquina y el sistema fallara en ese instante, la máquina quedaría en estado *reservada* sin ninguna reserva activa asociada. Garantizar el orden correcto reduce el riesgo de estados inconsistentes.

3.11. `BookingConfirmService` `confirm(String)`

Permite a un estudiante confirmar su asistencia al turno reservado cuando está dentro de la ventana de confirmación.

Casos de prueba

- TC1. **Reserva activa y dentro de la ventana** confirmación exitosa.
- TC2. **[Límite] Sin reservas activas hoy** `IllegalStateException` con el número de documento en el mensaje.
- TC3. **[Límite] Reserva activa pero fuera de la ventana** `IllegalStateException` con los minutos de la ventana en el mensaje.
- TC4. **Orden de operaciones** la confirmación se delega en el helper antes de persistir.

Relevancia de TC2 y TC3: los mensajes de error contextualizados (número de documento, minutos de ventana) son especialmente importantes en una API REST consumida por un frontend: permiten mostrar al usuario exactamente por qué no puede confirmar, evitando llamadas de soporte innecesarias.

3.12. `BookingConfirmService releaseExpiredSlots`

Tarea programada que libera automáticamente las máquinas cuya reserva confirmada ya finalizó.

Casos de prueba

- TC1. **Sin reservas hoy** no se ejecuta ninguna acción de liberación.
- TC2. **Reserva confirmada con turno expirado y sin siguiente reserva** la máquina se libera.
- TC3. **[Límite] Reserva confirmada con turno expirado pero hay siguiente reserva** la máquina *no* se libera.
- TC4. **[Límite] Reserva activa (no confirmada) con turno expirado** la máquina *no* se libera.
- TC5. **[Límite] Reserva confirmada cuyo turno aún no ha terminado** la máquina *no* se libera.
- TC6. **Múltiples reservas mixtas** solo se liberan las máquinas elegibles; las demás permanecen intactas.

Relevancia de TC3 y TC4: TC3 evita liberar una máquina que en realidad está siendo usada por el siguiente turno. TC4 protege contra la liberación accidental de una máquina reservada cuyo usuario aún no confirmó asistencia: en ese caso la reserva sigue activa (no confirmada) y no debe liberarse hasta que el sistema la gestione por otra vía.

3.13. `BookingConfirmHelper findTodayActiveBookings`

Recupera las reservas del día actual en estado *activo* para un estudiante dado.

Casos de prueba

- TC1. **Existe una reserva activa para hoy** se retorna en la lista.
- TC2. **El estudiante no tiene reservas** lista vacía.
- TC3. **[Límite] Reserva activa pero de otro día** excluida de la lista.
- TC4. **[Límite] Reserva de hoy pero cancelada** excluida de la lista.

Relevancia de TC3 y TC4: el método debe aplicar *dos* filtros simultáneamente (fecha = hoy y estado = activo). Un error en cualquiera de los dos permitiría confirmar

reservas del pasado o reservas ya canceladas.

3.14. `BookingConfirmHelper` `findBookingReadyToStart`

Busca, dentro de una lista de reservas activas, aquella cuyo turno está dentro de la ventana de confirmación (los N minutos previos al inicio).

Casos de prueba

- TC1. Reserva dentro de la ventana se retorna.
- TC2. **[Límite]** Turno que inicia exactamente ahora se retorna (límite inclusivo).
- TC3. **[Límite]** Turno demasiado lejano en el futuro null; aún no es momento de confirmar.
- TC4. **[Límite]** Turno ya iniciado null; la ventana ha expirado.
- TC5. Lista vacía null.
- TC6. Varias reservas; solo una en la ventana se retorna la correcta.

Relevancia de TC2 y TC4: el caso exactamente en el límite (TC2) prueba que la condición de comparación es $\leq$ y no $<$. TC4 asegura que un turno ya iniciado no pueda confirmarse retroactivamente, evitando abusos.

3.15. `BookingConfirmHelper` `hasNextBooking`

Comprueba si la máquina de una reserva tiene otra reserva activa en el turno inmediatamente siguiente, con el fin de no liberar la máquina innecesariamente.

Casos de prueba

- TC1. Siguiendo turno existe y está reservado true.
- TC2. **[Límite]** Siguiendo turno no existe false (era el último turno del día).
- TC3. Siguiendo turno existe pero sin reservas false.
- TC4. **[Límite]** Turno nulo en la reserva false de forma segura.

Relevancia de TC2: cuando una reserva corresponde al último turno disponible, no existe un turno $n + 1$. Sin esta prueba, el método podría fallar al intentar buscar un ID inexistente.